

TECHNICAL REPORT

DESIGN AND IMPLEMENTATION OF THE HYBRID FTP APPLICATION

Reliable File Transfer over TCP Control and UDP Data Channels

Project Information	Details
Course	Internetworking Protocol
Member A	Nguyen Hoang Phat -23125066
Member B	Le Minh Qui - 23125069
Repository	https://github.com/lmqui23/hybrid-ftp-system

Implementation language: Python 3 | Transport APIs: native socket library

Executive Summary

This report presents a Hybrid FTP client-server application that separates the control plane from the data plane. FTP commands, replies, authentication state, transfer identifiers, file sizes, and SHA-256 digests travel over TCP. Actual file payloads travel over UDP through a custom Stop-and-Wait reliable data transfer (RDT) protocol implemented from first principles.

The RDT layer provides sequence numbers, acknowledgements, CRC32 corruption detection, timeout-based retransmission, duplicate elimination, cancellation, temporary-file safety, and end-to-end SHA-256 verification. A sender window of one packet bounds in-flight data and prevents network flooding. The application supports binary transfers, Active and Passive modes, multiple concurrent TCP sessions, directory operations, and standard three-digit FTP replies.

Verification includes twelve automated protocol, RDT, and end-to-end integration tests and live end-to-end demonstrations of Passive upload, Active upload, Active download, directory listing, append, unique upload, and cross-host hash equality.

Document Map

Section	Purpose
1	Application scenario and TCP/UDP protocol interaction
2	Project-wide data structures and wire formats
3	Functional workflows and state machines
4	Task assignment matrix
5	Self-assessment and peer evaluation
6	GenAI usage and code refinement log
7	Application demo evidence

1. Application Scenario and Protocol Interaction

1.1 System Objective

The application mirrors the architectural principle of RFC 959 by using a persistent TCP control connection and separate, short-lived UDP data sockets. TCP preserves command ordering and session state. UDP carries all file and listing payloads, while the custom RDT layer restores the reliability properties required for correct file transfer.

1.2 Component Architecture

Component	Primary responsibility	Representative modules
TCP Client	CLI, FTP commands, reply parsing, PASV/PORT negotiation	client/control_client.py
TCP Server	Authentication, command dispatch, path validation, session threads	server/tcp_server.py
FTP Adapter	Maps FTP transfers to RDT sender/receiver calls	common/session.py; common/data_transfer.py
RDT	Stop-and-Wait states, ACKs, timeout, retransmission, cancellation	rdt/sender.py; rdt/receiver.py
Protocol	32-byte header, encoding, CRC32, SHA-256	protocol/packet.py; protocol/codec.py

1.3 Full Download Lifecycle

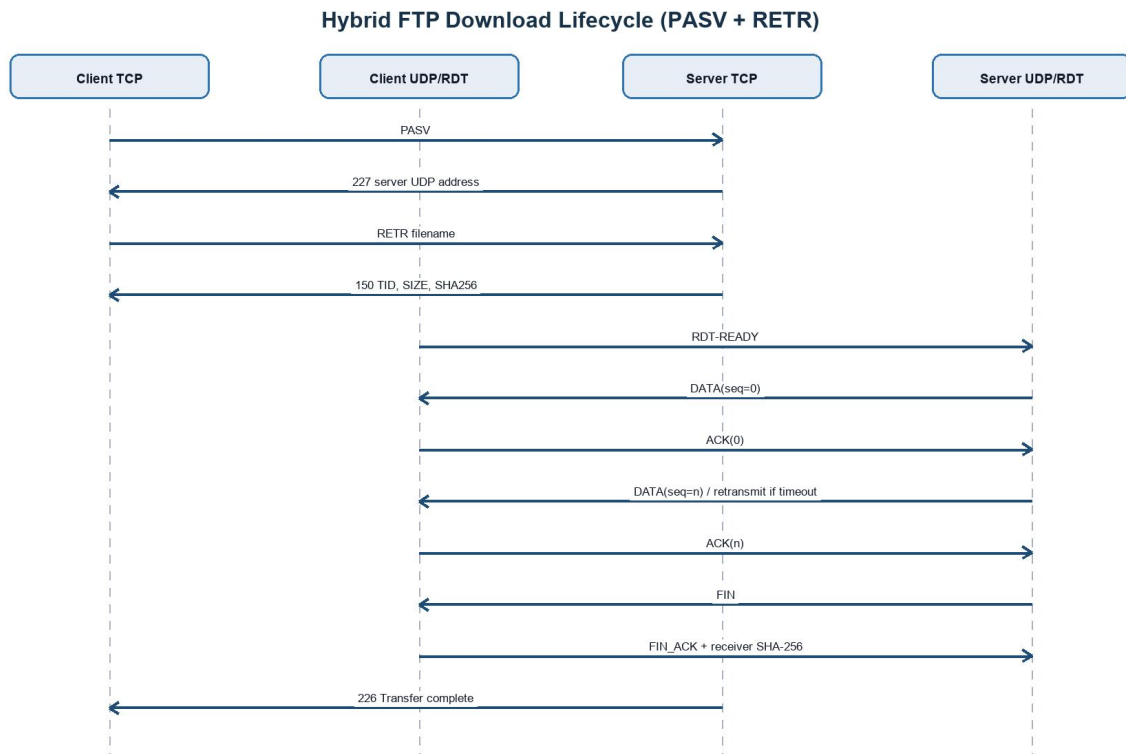


Figure 1. TCP control and UDP RDT interaction for a Passive-mode download.

The 150 reply is the synchronization boundary. It carries the transfer ID, exact size, and expected hash before either RDT endpoint processes file payload. The server issues 226 only after the sender receives FIN_ACK and verifies that the receiver-calculated hash equals the source hash.

1.4 Control and Data Separation

TCP control channel	UDP data channel
USER/PASS and session commands	DATA packet payload
PASV/PORT negotiation	ACK and retransmitted DATA
Transfer ID, size, SHA-256	FIN, FIN_ACK, CANCEL
150/226/425/426 replies	CRC32-protected RDT datagrams

2. Project-Wide Data Structures

2.1 TCP Command and Reply Format

Commands and replies are UTF-8 text terminated by CRLF. The base command grammar is:

```
COMMAND [argument] CRLF
REPLY_CODE SP message CRLF
```

For a data transfer, the preliminary reply extends the message with machine-readable metadata:

```
150 TID=<uint64> SIZE=<bytes> SHA256=<64-hex-characters> [FILE=<name>] CRLF
```

For uploads, the current client appends SIZE and SHA256 fields to STOR, APPE, or STOU over TCP. No file payload or RDT metadata packet is sent through UDP.

2.2 UDP Custom Header

All multibyte fields use network byte order. The fixed header is 32 bytes, followed by zero to 1,024 payload bytes.

Offset	Bytes	Field	Purpose
0	4	Magic	0x52445431 (ASCII RDT1) identifies the protocol
4	1	Version	Wire-format compatibility; current value 1
5	1	Flags	DATA, ACK, FIN, FIN_ACK, ERROR, CANCEL
6	2	Header size	Constant 32 bytes
8	8	Transfer ID	Separates concurrent or stale transfers
16	4	Sequence number	Orders DATA and FIN packets

20	4	ACK number	Identifies the acknowledged sequence
24	2	Advertised window	Fixed at one for Stop-and-Wait flow control
26	2	Payload length	Exact payload bytes in this datagram
28	4	CRC32	Detects header or payload corruption

The CRC32 is computed over the complete header with its checksum field set to zero, concatenated with the payload. A receiver discards a datagram when its calculated CRC32 differs from the transmitted value.

2.3 Packet Flags and Semantics

Flag	Direction	Meaning
DATA	Sender -> Receiver	Carries one ordered file chunk
ACK	Receiver -> Sender	Confirms one DATA sequence number
FIN	Sender -> Receiver	Marks the end of the stream
FIN_ACK	Receiver -> Sender	Confirms FIN and returns receiver SHA-256
CANCEL	Either direction	Cooperatively aborts the active transfer
ERROR	Reserved	Protocol-level error signalling for extension

2.4 Session and Transfer Structures

Structure	Important fields	Role
FTPSession	authentication, root/current directory, mode, UDP target, active context	Maintains one client's TCP and data-channel state
TransferContext	transfer_id, state, config, cancel_event, statistics	Owens one independent RDT operation
RDTConfig	payload_size, RTO, idle timeout, retry limit, poll interval	Controls reliability and timing
TransferStatistics	bytes, packets, ACKs, retransmissions, duplicates, corruption, duration	Supports progress, evaluation, and debugging
RDTPacket	flags, IDs, sequence/ACK numbers, window, payload, checksum	In-memory representation of a UDP datagram

3. Functional Workflows

3.1 Server Thread Dispatch

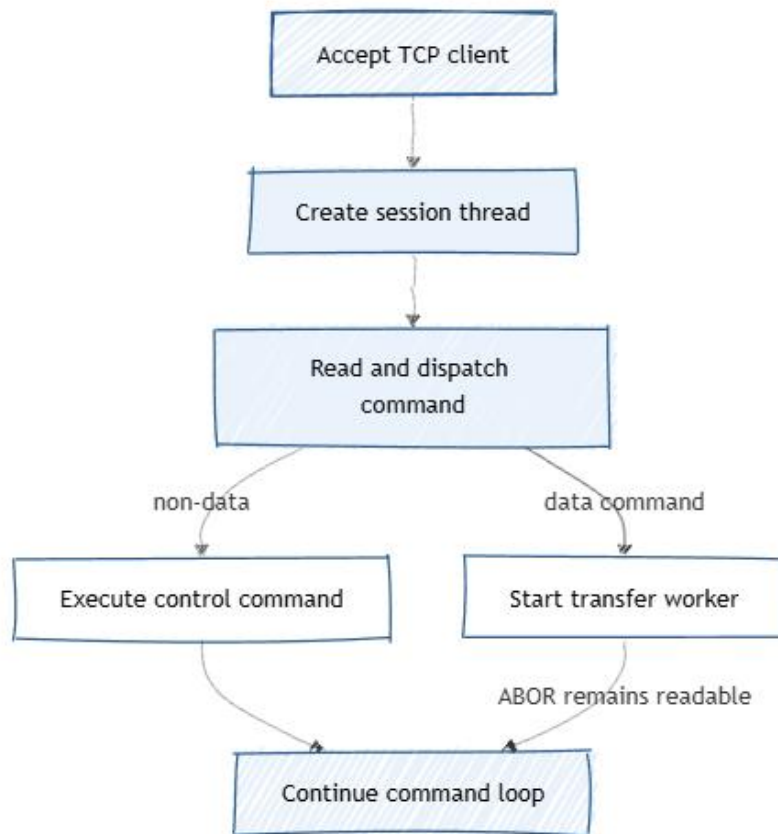


Figure 2. Per-client TCP session and independent data-transfer worker.

Each accepted TCP client receives an isolated FTPSession and daemon thread. Data commands start a second worker so the control thread can continue receiving ABOR, NOOP, or QUIT. While a transfer is active, unrelated commands receive a transient failure response.

3.2 Stop-and-Wait Sender State Machine

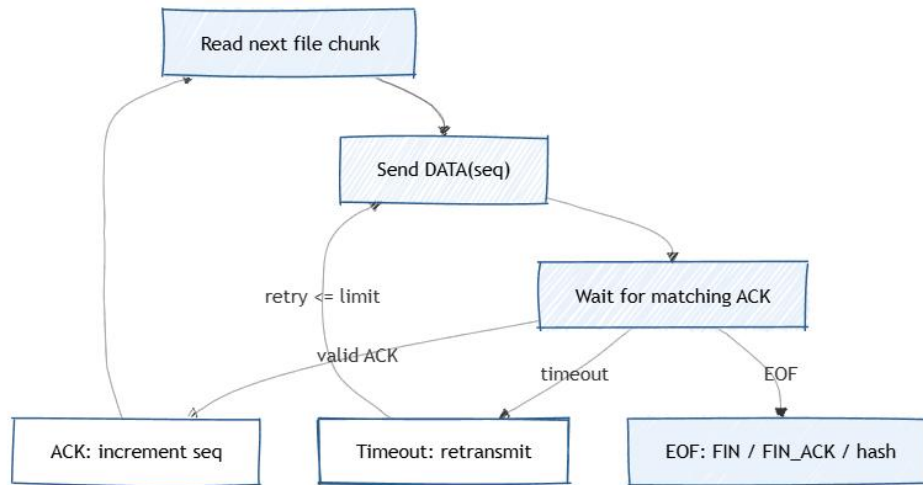


Figure 3. Sender state transitions.

At most one DATA packet is unacknowledged. Although a dynamic sliding window protocol (such as Go-Back-N or Selective Repeat) is not implemented, fixing the sender window size to 1 packet serves as a fundamental Stop-and-Wait flow control mechanism. This guarantees that in-flight data never exceeds a single packet, effectively preventing receiver buffer overflow and network flooding. While this trades throughput on high-delay links for implementation simplicity, it strictly satisfies the flow-control requirement.

3.3 Stop-and-Wait Receiver State Machine

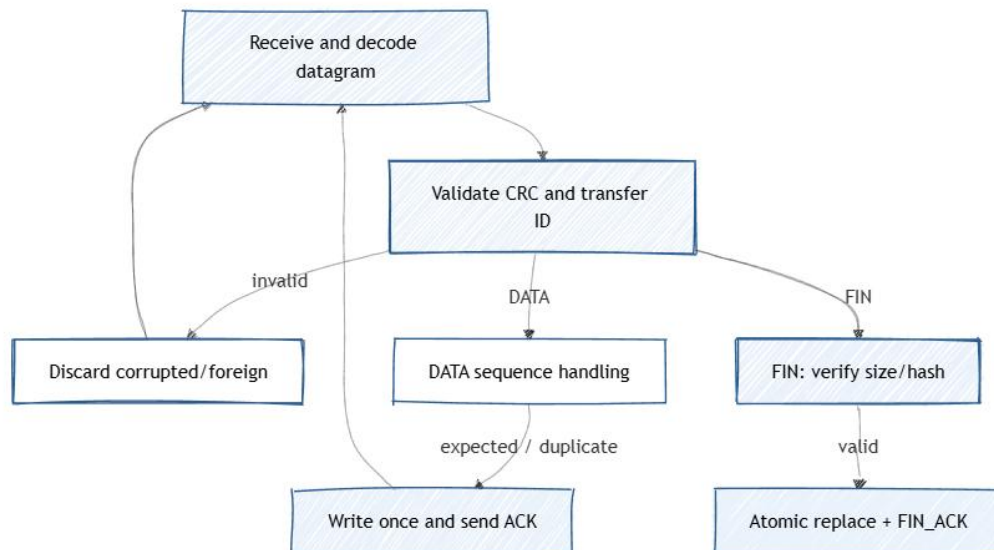


Figure 4. Receiver validation, duplicate handling, and safe commit.

3.4 Active and Passive Mode

Aspect	Passive mode (PASV)	Active mode (PORT)
UDP listener	Server binds an ephemeral UDP port	Client binds an ephemeral UDP port
TCP negotiation	Server returns 227 with IP and port	Client sends its IP and port
Peer discovery	Client sends RDT-READY when server sends	Server sends RDT-READY when client sends
Typical network behavior	More NAT/firewall friendly	Demonstrates server-initiated data contact
Socket lifetime	One transfer	One transfer

3.5 Upload, Download, Listing, and Cancellation

- RETR: server is the RDT sender; client is the receiver.
- STOR/APPE/STOU: client is the sender; server is the receiver.
- LIST/NLST: server serializes the listing to bytes and sends it through the same verified RDT path.
- ABOR: the TCP handler sets the active context's cancellation event; the RDT endpoint emits CANCEL and removes incomplete temporary data.
- APPE: the verified uploaded object is appended only after RDT reception succeeds.

4. Task Assignment Matrix

Replace the bracketed identity fields before submission. The ownership split below reflects the repository architecture and should be adjusted only if the actual work history differs.

Engineering component	Owner	Collaborator	Evidence / modules
TCP control server and FTP reply handling	Le Minh Qui	Nguyen Hoang Phat review	server/tcp_server.py
Client CLI and command dispatch	Le Minh Qui	Nguyen Hoang Phat review	client/control_client.py
Authentication and filesystem operations	Le Minh Qui	-	server/file_system.py
Active/Passive session setup	Le Minh Qui	Nguyen Hoang Phat integration support	common/session.py;common/data_transfer.py
UDP packet header and codec	Nguyen Hoang Phat	Le Minh Qui review	protocol/
Stop-and-Wait sender and receiver	Nguyen Hoang Phat	Le Minh Qui integration support	rdt/sender.py; rdt/receiver.py
CRC32 and SHA-256 verification	Nguyen Hoang Phat	Le Minh Qui TCP metadata	protocol/crc32.py; protocol/file_hash.py
Fault injection and automated RDT tests	Nguyen Hoang	-	rdt/fault_injector.py; tests/

	Phat		
End-to-end integration and demo	Both	Both	common/ftp_shared.py and live logs
Technical report and presentation	Both	Both	This document and presentation materials

5. Self-Assessment and Peer Evaluation

Le Minh Qui: Implemented the TCP control server, command handling, multi-threaded session isolation, filesystem management, and Active/Passive mode negotiation (PORT/PASV). Addressed integration challenges by ensuring data transfers run on background threads so the TCP control channel remains responsive to concurrent commands like ABOR.

Nguyen Hoang Phat: Designed the 32-byte UDP packet header and implemented the Stop-and-Wait RDT mechanism, including CRC32 integrity checks, sequence/ACK tracking, retransmissions, duplicate filtering, temporary file commits, and end-to-end SHA-256 verification. Maintained a clean API boundary between the RDT transport layer and FTP logic.

Peer Evaluation and Contribution Declaration

Member	Declared contribution
Nguyen Hoang Phat - 23125066	50%
Le Minh Qui - 23125069	50%
Total	100%

6. GenAI Usage and Code Refinement Log

This document records the proposed prompts, condensed AI responses, human review and refinement, and verification results for the Hybrid FTP project.

The prompts below are standardized engineering prompts. Before submission, The prompts below are standardized engineering prompts. They must not be described as original prompts unless they match the exported conversation.

Session 1 — Requirement Analysis, Project Structure, and Work Allocation

Date: 2026-07-23

Tool: ChatGPT

Objective: Analyze the complete assignment, inspect the repository, propose the directory structure, and prepare an independent plan for two members.

Prompt 1 — Analyze the Assignment and Evaluation Criteria**Proposed prompt sent to AI:**

Act as a senior software architect and networking lecturer.

Read the complete assignment specification and inspect the current repository before proposing any implementation.

Project context:

- The application is a Hybrid FTP system.
- TCP must carry authentication, FTP commands, replies, session state, and transfer metadata.
- File and directory-listing payloads must be transferred over UDP.
- Reliability over UDP must be implemented by the team.
- The implementation language is Python 3.
- Third-party FTP, reliable-UDP, QUIC, KCP, and file-transfer frameworks are prohibited.

Tasks:

1. Extract every functional and non-functional requirement from the specification.
2. Separate mandatory requirements from optional or Excellent-level evidence.
3. Identify requirements for TCP, UDP, reliability, integrity, concurrency, Active/Passive mode, cancellation, security, testing, documentation, and demonstration.
4. Create a requirement-to-evidence matrix.
5. Identify ambiguous requirements and state conservative assumptions.
6. List technical risks that could cause the project to fail during testing or oral defense.

Do not write code yet.

Expected output:

- requirement matrix;
- acceptance criteria;
- risk list;
- assumptions requiring human confirmation;
- recommended implementation order.

Raw output (condensed):

The AI divided the requirements into the following groups:

- TCP control: authentication, command parsing, reply codes, and sessions;
- UDP data: DATA, ACK, timeout, retransmission, and transfer completion;
- integrity: CRC32 per packet and SHA-256 for the complete file;

- data modes: PASV and PORT;
- safety: path traversal prevention, temporary files, and cleanup;
- concurrency: multiple clients, transfer workers, and ABOR;
- verification: unit tests, integration tests, fault injection, and live demonstrations.

The response recommended that success should not be demonstrated by a single file transfer. Evidence should also cover loss, corruption, duplication, timeout, cancellation, concurrency, and integrity failure.

Analysis and refinement:

- The TCP-control and UDP-data grouping was retained because it matches the assignment architecture.
- TCP was explicitly required to carry `transfer_id`, exact size, and SHA-256 before UDP transfer begins.
- Stop-and-Wait was documented as flow control with a window of one, not as adaptive congestion control.
- Destination preservation after a failed transfer was added as an acceptance criterion.
- Requirements not supported by the rubric were not added unnecessarily.

Prompt 2 — Inspect the Repository and Propose the Directory Structure

Proposed prompt sent to AI:

Inspect the complete repository recursively.

Before changing any file:

1. List every source, test, documentation, storage, and generated-artifact file.
2. Identify the language and purpose of each implementation file.
3. Detect placeholder, legacy, duplicate, or unused files.
4. Map imports and dependencies between modules.
5. Identify entry points for the server, client, and tests.
6. Determine which components already exist and which are missing.

Then propose a minimal project structure for a two-person Python networking assignment.

Architecture constraints:

- protocol/ owns the packet model, binary codec, CRC32, and file hashing.
- rdt/ owns Stop-and-Wait configuration, context, sender, receiver, and fault injection.
- common/ owns shared FTP session state and TCP-to-RDT transfer orchestration.
- client/ owns the interactive FTP client and TCP reply handling.
- server/ owns the TCP server and safe filesystem operations.

- tests/ contains protocol, RDT, command, and end-to-end tests.
- protocol/ and rdt/ must not import client/ or server/.
- Client and server must not duplicate reliable-UDP logic.

Expected output:

1. Current repository tree.
2. Proposed repository tree.
3. Responsibility of every proposed file.
4. Allowed dependency direction.
5. Files to retain, move, split, or remove, with justification.
6. A migration plan that preserves existing behavior.

Raw output (condensed):

The AI proposed:

client/ control_client.py

server/ tcp_server.py;file_system.py

common/session.py; data_transfer.py

protocol/ packet.py; codec.py; crc32.py; file_hash.py

rdt/ config.py; context.py; sender.py; receiver.py; fault_injector.py

tests/ test_protocol.py; test_rdt.py; test_ftp_commands.py; test_ftp_integration.py

Recommended dependency direction:

protocol -> rdt -> common -> client/server -> integration tests

Analysis and refinement:

- protocol/ and rdt/were kept independent from the FTP control layer.
- FTPSession, DataMode, andTransferTypewere separated into

common/session.py.

- Active/Passive setup and the RDT adapter remained in

common/data_transfer.py.

- client/control_client.py was simplified to

client/control_client.py.

- server/ was flattened into files directly under server/.
- Legacy C++ placeholders were excluded from the active Python structure.

Prompt 3 — Divide the Work Between Two Members**Proposed prompt sent to AI:**

Act as the technical lead for a two-member networking project.

Using the approved architecture, divide the work into two independently implementable parts:

- Member 1: TCP control plane and FTP application behavior.
- Member 2: custom reliable UDP protocol and transport behavior.

Requirements:

1. Assign explicit file ownership to each member.
2. Define the public interface between both parts before implementation.
3. Prevent both members from editing the same core files during independent development.
4. Assign unit tests to the owner of each module.
5. Assign integration, end-to-end testing, documentation, and demonstration to both members.
6. Define the merge order and review responsibilities.
7. Ensure each member has a substantial design and implementation contribution.

Expected output:

- ownership matrix;
- deliverables for each member;
- shared integration contract;
- milestone plan;
- review checklist;
- risks caused by parallel development.

Raw output (condensed):

Component	Primary owner	Collaboration
TCP server and FTP replies	Member 1	Member 2 reviews integration
Client control and CLI	Member 1	Member 2 supports the data API
Authentication and filesystem	Member 1	Both review security
Active/Passive negotiation	Member 1	Member 2 confirms UDP contract
Packet model and codec	Member 2	Member 1 reviews metadata

Stop-and-Wait sender/receiver	Member 2	Member 1 supports integration
CRC32, SHA-256, fault injection	Member 2	Both prepare demonstrations
Integration, ABOR, concurrency	Both	Both
Complete testing and report	Both	Both

Analysis and refinement:

- Member 1 does not reimplement timeout, retransmission, or packet ordering.
- Member 2 does not modify authentication, FTP commands, or filesystem authorization.
- Both members agree on transfer ID, size, SHA-256, UDP endpoint, cancellation, and transfer-result contracts before integration.
- Part B is stabilized independently before Part A calls it through its public API.

Session 1 Results

Item	Result
Requirement analysis	Requirement groups and acceptance criteria defined
Directory structure	`protocol`, `rdt`, `common`, `client`, and `server` separated
Work allocation	Two independent parts with an explicit integration contract
Code changes	No implementation changes during the initial analysis

Session 2 — Prompts for the TCP Member**Member:**Le Minh Qui - 23125069**Date:**2026-07-24**Tool:**ChatGPT**Scope:**client/,server/,common/session.py**Prompt 1 — Design the TCP Control Server****Proposed prompt sent to AI:**

Act as a senior Python network engineer responsible only for the FTP control plane.
Inspect the current repository and design server/tcp_server.py.

Requirements:

- Listen for TCP clients on a configurable control port.
- Use one isolated session per client.
- Parse CRLF-terminated commands from a TCP byte stream.
- Do not assume one recv() call equals one command.

- Support authentication and reject invalid command order.
- Produce standard three-digit FTP replies and multiline HELP replies.
- Support all FTP commands required by the assignment.
- Run data transfers in worker threads so the control loop can receive ABOR.
- Serialize replies from the control thread and transfer worker.
- Maintain a thread-safe active-session table.
- Delegate file payload transfer to common/data_transfer.py.

Constraints:

- Do not implement packet encoding, ACK handling, timeout, or retransmission.
- Do not send file payload bytes over TCP.
- Preserve existing public RDT interfaces.

Expected output:

1. Session state design.
2. Command-to-handler matrix.
3. Reply-code mapping.
4. Thread and locking model.
5. Implementation changes.
6. Unit tests.

Raw output (condensed):

The AI proposed:

- one thread per TCP client;
- CRLF-oriented command reading;
- an isolated FTPSession;
- a reply lock to prevent response interleaving;
- a separate transfer worker;
- 150before transfer,226on success,425on setup failure, and426 on cancellation or transfer failure.

Analysis and refinement:

- Thread-per-client was retained because it is simple and suitable for the assignment scale.
- Command-order validation was added before authentication.
- The command loop remains responsive while the transfer worker runs.

- RDT implementation details remain outside tcp_server.py.

Prompt 2 — Build Session and Active/Passive Control

Proposed prompt sent to AI:

Design the FTP session model and the TCP-side Active/Passive negotiation.

The session must track:

- control connection identity;
- authentication and username;
- server root and current directory;
- transfer type;
- selected data mode;
- passive UDP socket or active target;
- active transfer context;
- transfer state and cancellation;
- synchronization primitives.

Requirements:

- PASV binds an ephemeral UDP port and returns a valid 227 reply.
- PORT validates six numeric fields and stores the advertised client endpoint.
- Data mode resets after each transfer.
- ABOR cancels the active context without blocking the TCP command loop.
- Socket cleanup must occur after success, error, cancellation, or disconnect.

Return:

- session fields and invariants;
- PASV/PORT state transitions;
- cleanup rules;
- implementation;
- negative test cases.

Raw output (condensed):

The AI proposed FTPSession, DataMode, and TransferType, including a passive socket, active IP/port, transfer context, abort flag, and transfer lock.

Analysis and refinement:

- The session model was separated from transfer orchestration.

- A passive socket is valid for one transfer only.
- The data mode is reset after transfer to prevent reuse of a stale endpoint.
- ABOR calls `TransferContext.cancel()`.

Prompt 3 — Filesystem Safety and FTP File Commands

Proposed prompt sent to AI:

Review and implement `server/file_system.py` and every filesystem-facing FTP command.

Requirements:

- Resolve all user paths relative to the configured server storage root.
- Reject path traversal and access outside that root.
- Support directory creation, removal, navigation, and listing.
- Support file size, modification time, SHA-256, delete, rename, append, and unique-name generation.
- Read large files in chunks when hashing.
- Distinguish files from directories.
- Do not replace a valid destination with incomplete upload data.

Commands to review:

PWD, CWD, CDUP, MKD, RMD, LIST, NLST, SIZE, MDTM, HASH, DELE, RNFR, RNTD, RETR, STOR, APPE, and STOU.

Return:

- security findings;
- safe-path algorithm;
- command behavior;
- implementation changes;
- path traversal and failure tests.

Raw output (condensed):

The AI proposed centralized safe-path resolution, chunked hashing, unique-name generation, and file/directory checks before every operation.

Analysis and refinement:

- Every path is normalized and checked against the server root.
- Upload data is staged before replacement or append.
- RNTD is valid only after RNFR.

Prompt 4 — TCP Client and CLI**Proposed prompt sent to AI:**

Implement client/control_client.py for the Hybrid FTP application.

Requirements:

- Connect to the TCP control server.
- Parse single-line and multiline FTP replies.
- Provide an interactive CLI for all supported commands.
- Track the selected PASV or PORT mode.
- Enter Passive mode automatically when a data command is issued without a selected mode.
- Parse TID, SIZE, and SHA256 from the 150 reply.
- Calculate local upload size and SHA-256 before STOR, APPE, or STOU.
- Start data transfer in a worker thread.
- Keep ABOR available while the worker is active.
- Delegate UDP transfer to common/data_transfer.py.
- Handle disconnect, malformed replies, and missing local files safely.

Do not implement the Stop-and-Wait protocol in the client module.

Return:

- client state model;
- reply parsing algorithm;
- CLI command flow;
- implementation;
- client and integration test cases.

Raw output (condensed):

The AI proposed FTPClient, a buffered reply reader, a metadata parser, automatic Passive mode, a background transfer thread, and explicit ABOR handling.

Analysis and refinement:

- The reply reader handles both code message and code-...multiline formats.
- The client does not know packet or retransmission details.
- Upload and download paths use client storage by default.

Session 2 Test Result

python -m unittest tests.test_ftp_commands -v

Command tests cover authentication, invalid command order, data modes, metadata, filesystem operations, path traversal, and reply codes.

Session 3 — Prompts for the UDP Member

Member:Nguyen Hoang Phat - 23125066

Date:2026-07-24

Tool:ChatGPT

Scope:protocol/,rdt/

Prompt 1 — Design the Packet and Wire Format

Proposed prompt sent to AI:

Act as a network protocol engineer.

Design a deterministic binary packet format for a Stop-and-Wait reliable UDP protocol.

Requirements:

- Fixed 32-byte header.
 - Network byte order.
 - Fields: magic, version, flags, header size, 64-bit transfer ID, sequence number, acknowledgment number, advertised window, payload length, CRC32.
 - Flags: DATA, ACK, FIN, FIN_ACK, ERROR, CANCEL.
 - Maximum payload: 1,024 bytes.
 - Reject invalid magic, version, flags, ranges, payload types, and payload sizes.
- Keep the packet model separate from encoding, CRC32, and whole-file SHA-256.

Return:

- byte-offset table;
- field semantics;
- packet data model;
- validation rules;
- unit-test cases;
- compatibility limitations.

Raw output (condensed):

The AI proposed an `RDTPacket` data class, a `PacketFlag` bitmask, a 32-byte header, and validation for every numeric range.

Analysis and refinement:

- Magic `0x52445431` and version `1` were selected.
- The advertised window is fixed at one.
- A 64-bit transfer ID isolates concurrent transfers and stale packets.

Prompt 2 — Encode, Decode, and CRC32

Proposed prompt sent to AI:

Implement `encode_packet()` and `decode_packet()` for the approved packet model.

Requirements:

- Use Python struct with network byte order.
- Encode a fixed 32-byte header.
- Calculate CRC32 over the header with checksum set to zero plus the payload.
- Reject truncated datagrams.
- Reject unknown flag bits.
- Reject invalid header size and oversized payload.
- Require exact equality between datagram length and `header_size + payload_length`.
- Reject corrupted header or payload before returning a packet.
- Use focused `PacketEncodeError` and `PacketDecodeError` exceptions.

Return:

- implementation;
- validation order;
- error behavior;
- round-trip and corruption tests.

Raw output (condensed):

The AI proposed format `!IBBHQIIHHI` , a zero-checksum header, CRC32 recomputation, and strict datagram-length equality.

Analysis and refinement:

- Trailing bytes are not accepted.
- A corrupted packet is ignored so the sender retransmits after timeout.
- CRC32 and SHA-256 remain separate.

Prompt 3 — Stop-and-Wait Sender

Proposed prompt sent to AI:

Implement a Stop-and-Wait UDP sender.

Requirements:

- Read the source file in configurable chunks.
- Keep exactly one unacknowledged DATA packet.
- Match replies by peer address, transfer ID, expected flag, and ACK number.
- Retransmit the same packet after timeout.
- Enforce a maximum retry count.
- Ignore unrelated and malformed datagrams.
- Complete with FIN and FIN_ACK.
- Compare the receiver SHA-256 returned in FIN_ACK with the source SHA-256.
- Support cooperative cancellation.
- Record bytes, packets, ACKs, retransmissions, corruption, and duration.

Do not claim that Stop-and-Wait is adaptive congestion control.

Return:

- sender state flow;
- implementation;
- failure behavior;
- deterministic tests;
- performance limitation.

Raw output (condensed):

The AI proposed a `shared_send_and_wait()` flow for DATA and FIN, deadlines based on `time.monotonic()`, bounded retries, and peer/transfer-ID filtering.

Analysis and refinement:

- The retry loop was kept in one method.
- CANCEL propagation was added.
- The sender completes only when the hash in FIN_ACK matches the source hash.

Prompt 4 — Stop-and-Wait Receiver and Fault Injection

Proposed prompt sent to AI:

Implement the matching Stop-and-Wait receiver and deterministic fault injection.

Receiver requirements:

- Accept one peer and the expected transfer ID.
- Write an in-order DATA packet exactly once.
- Re-ACK duplicate packets without writing duplicate bytes.
- Do not advance for an out-of-order packet.
- Enforce the size announced through TCP.
- Write into `.part.<transfer_id>` temporary storage.
- Verify SHA-256 before replacing the destination.
- Preserve an existing destination after failure.
- Remove temporary files after timeout, cancellation, or integrity failure.
- Re-send FIN_ACK when FIN is duplicated.

Fault injector requirements:

- Simulate packet loss, one-bit corruption, and duplication.
- Accept a random seed for deterministic tests.
- Disable every fault rate by default.

Return:

- receiver state flow;
- implementation;
- cleanup guarantees;
- fault model;
- tests for all failure paths.

Raw output (condensed):

The AI proposed expected-sequence tracking, duplicate re-ACK, a temporary file, atomic replacement, FIN_ACK lingering, and a seeded FaultInjector.

Analysis and refinement:

- The receiver filters transfer ID before selecting a peer.
- The temporary file is removed on every exception path.
- An existing destination is replaced only after size and SHA-256 verification.

Session 3 Test Result

```
python -m unittest tests.test_protocol tests.test_rdt -v
```

Tests cover text, binary and empty files, corruption, loss, duplication, out-of-order packets, wrong transfer IDs, retry exhaustion, lost FIN_ACK, cancellation, and hash failure.

Session 4 — TCP and UDP Integration

Date:2026-07-24

Tool:ChatGPT

Responsible members:Both

Objective:Connect Part A and Part B through a public interface without mixing responsibilities.

Prompt 1 — Design the Integration Contract

Proposed prompt sent to AI:

Act as the integration engineer for two independently developed components.

Part A owns:

- TCP control;
- authentication and FTP commands;
- filesystem validation;
- PASV/PORT negotiation;
- transfer metadata and FTP replies.

Part B owns:

- packet encoding and validation;
- Stop-and-Wait sender and receiver;
- timeout, retransmission, cancellation, and integrity verification.

Design the integration through common/data_transfer.py.

Requirements:

- Exchange transfer ID, exact size, and SHA-256 through TCP before UDP data.
- Carry every file and listing payload byte through UDP.
- Map RETR, STOR, APPE, STOU, LIST, and NLST to sender/receiver roles.
- Support Passive and Active modes.
- Use a readiness datagram only for UDP peer discovery.
- Allocate an isolated transfer context and socket per transfer.
- Keep ABOR available through the TCP command loop.

- Close sockets and reset data mode after success or failure.
- Convert setup and transfer results into 150, 226, 425, and 426 replies.

Return:

- public API;
- command-to-role matrix;
- upload/download sequence;
- error mapping;
- cleanup rules;
- integration order.

Raw output (condensed):

Command	Sender	Receiver
`RETR`	Server	Client
`STOR`, `APPE`, `STOU`	Client	Server
`LIST`, `NLST`	Server	Client

Preliminary metadata:

150 TID=<uint64> SIZE=<bytes> SHA256=<64-hex-digest>

Analysis and refinement:

- Metadata remains on TCP because the receiver needs size and hash before UDP reception.
- The readiness datagram is not counted as file payload.
- Uploads are staged before replacement or append.
- Socket cleanup occurs in finally blocks.

Prompt 2 — Implement Integration and ABOR**Proposed prompt sent to AI:**

Implement the approved TCP-to-RDT integration.
--

Required workflow:

1. Implement Passive RETR.
2. Implement Passive STOR.
3. Verify size and SHA-256 equality.
4. Add LIST and NLST through the same RDT path.
5. Add Active mode using PORT.
6. Add APPE and STOU with safe staging.

7. Run transfers in workers.
8. Connect ABOR to the active TransferContext.
9. Reset session data mode and close every data socket.
10. Add integration tests after each group.

Do not rewrite protocol/ or rdt/ unless an interface defect is demonstrated by a failing test.

Return:

- files changed;
- implementation summary;
- observed failures and fixes;
- final integration tests;
- remaining limitations.

Raw output (condensed):

The AI connected the client and server through common/data_transfer.py, created one transfer context per operation, added progress/retry statistics, and propagated ABOR.

Analysis and refinement:

- Part B is not changed without a failing test that demonstrates a defect.
- Reply locking prevents transfer workers and the command loop from interleaving responses.
- Both upload and download cancellation are tested.

Session 5 — Testing and Hardening

Date:2026-07-24

Tool:ChatGPT

Responsible members:Both

Objective:Audit coverage, add negative-path tests, and verify the complete system.

Prompt 1 — Audit Complete Test Coverage

Proposed prompt sent to AI:

Act as an independent verification engineer.

Inspect the assignment, source code, and complete test suite. Do not assume a feature works because one transfer succeeds.

Tasks:

1. Produce a requirement-to-test matrix.
2. Identify unsupported claims and missing negative paths.
3. Add deterministic tests without Internet dependencies.
4. Use short test-specific timeouts and retry limits.
5. Verify cleanup of sockets and temporary files.
6. Run compile checks and the full test suite.

Required coverage:

- authentication success and failure;
- invalid command order;
- FTP command and reply categories;
- path traversal and filesystem operations;
- packet round trip, bounds, and corruption;
- text, binary, and empty files;
- timeout, retransmission, loss, corruption, and duplication;
- wrong transfer ID and out-of-order packets;
- retry exhaustion and lost FIN_ACK;
- SHA-256 mismatch and destination preservation;
- PASV, PORT, upload, download, LIST, NLST, APPE, and STOU;
- two concurrent clients;
- live upload and download cancellation.

Expected output:

- coverage matrix;
- tests added or changed;
- exact commands executed;
- exact test count and observed duration;
- failures found and fixes applied;
- remaining limitations.

Raw output (condensed):

The AI divided the suite into:

- `test_protocol.py`: wire format and corruption;
- `test_rdt.py`: reliability, integrity, and cleanup;
- `test_ftp_commands.py`: authentication, commands, paths, and replies;
- `test_ftp_integration.py`: live transfers, concurrency, and ABOR.

Analysis and refinement:

- Lost FIN_ACK, total loss, and wrong transfer ID coverage were added.
- Tests verify that an existing destination survives a hash mismatch.
- Concurrent-client tests perform real transfers rather than only logins.
- Both upload and download ABOR are tested.
- Dynamic ports reduce binding conflicts during rapid test execution.

Final Verification Result

```
python -m compileall -q client common protocol rdt server tests
```

```
python -m unittest discover -s tests -v
```

Most recent observed result:

Ran 26 tests in 10.738s

OK

Execution time is environment-dependent and should be updated after the final pre-submission run.

AI Contribution and Human Refinement Summary

The percentages below summarize the recorded AI contribution and subsequent human refinement:

Area	AI contribution	Human refinement	Evidence
Requirement analysis	70%	30%	Requirement matrix
Project structure	75%	25%	Repository tree
Work allocation	60%	40%	Ownership matrix
TCP server and client	75%	25%	<code>`client/`, `server/`</code>
Packet and RDT	75%	25%	<code>`protocol/`, `rdt/`</code>
Integration	70%	30%	<code>`common/data_transfer.py`</code>

Automated tests	75%	25%	`tests/`
Documentation	50%	50%	`docs/`

7. Application Demo Evidence

7.1 Automated Verification

```
python -m unittest discover -s tests -p "test_*.py" -v
```

Ran 26 tests in 10.372s

OK

Test area	Verified behavior	Result
Packet round trip	Header encoding/decoding and binary payload preservation	PASS
Corruption	Modified datagram rejected by CRC32	PASS
Empty file	FIN handshake and empty SHA-256	PASS
Binary file	All byte values preserved	PASS
Network faults	Loss, corruption, duplicates, and retransmission	PASS
Transfer isolation	Wrong transfer ID ignored	PASS
Cancellation	Temporary file removed and state cancelled	PASS
Configuration	Invalid payload/timeout rejected	PASS
Total packet loss	100% loss reaches retry limit and marks transfer FAILED	PASS
Safe failure	Hash failure preserves the previous destination and removes .part	PASS
FTP integration	PASV/PORT, STOR/RETR, NLST, APPE, STOU, HELP, framing, two clients	PASS
Live ABOR	TCP ABOR cancels an 8 MiB download and removes temporary data	PASS

7.2 Live Integration Results

Scenario	Observed evidence	Result
----------	-------------------	--------

Authentication	USER admin / PASS produced 230	PASS
Passive upload	150 metadata followed by 226; 20,000-byte file stored	PASS
Active upload	PORT returned 200; source/server SHA-256 equal	PASS
Active download	RETR completed; client/server SHA-256 equal	PASS
NLST	Directory names delivered through verified UDP RDT	PASS
APPE	Server file size increased from 20,000 to 40,000 bytes	PASS
STOU	Server generated a unique stou_*.tmp filename	PASS

```

150 TID=<generated uint64> SIZE=20000 SHA256=5834dc0e...
226 Transfer complete.
Client SHA256: 5834dc0e2bb0de8ab148211525583c757407ce358afcd4d0358bd15c73f863b9
Server SHA256: 5834dc0e2bb0de8ab148211525583c757407ce358afcd4d0358bd15c73f863b9

```

7.3 Required Screenshot Placeholders

Insert original screenshots from the final clean-machine demonstration in the spaces listed below. Do not replace them with fabricated images.

Figure	Required screenshot	Status
A	Server startup, connected client IP, USER/PASS commands	<pre> [Client] Connecting to 127.0.0.1:2121... 220 Welcome to RDT-FTP Server (Hybrid TCP/UDP) ===== HYBRID FTP CLIENT CLI - READY FOR COMMANDS ===== [Auth/Session]: USER, PASS, QUIT [Data Mode] : PASV, PORT, TYPE [Directory] : PWD, CWD, CDUP, MKD, RMD [File/Data] : LIST, NLST, RETR, STOR, APPE, STOU, DELE, RNFR, RNTD, ABOR [Info/Meta] : STAT, SIZE, MDTM, HASH ===== ftp> USER admin 331 User name okay, need password. ftp> PASS 123456 [2026-07-25 07:58:41] [INFO] [SERVER] TCP Control Engine listening on port 2121... [ACTIVE SESSIONS] FD CLIENT USER MODE TRANSFER 4 127.0.0.1:37392 - NONE IDLE [2026-07-25 07:58:49] [INFO] [FD 4] Session initialized for 127.0.0.1:37392 [2026-07-25 07:58:56] [INFO] [FD 4] Command received: USER admin [2026-07-25 07:59:03] [INFO] [FD 4] Command received: PASS 123456 [2026-07-25 07:59:03] [INFO] [FD 4] User 'admin' authenticated successfully. [ACTIVE SESSIONS] FD CLIENT USER MODE TRANSFER 4 127.0.0.1:37392 admin NONE IDLE </pre>

B	Passive upload showing 227, 150 metadata, and 226	<pre> ftp> PASV 227 Entering Passive Mode (127,0,0,1,135,137) ftp> STOR sample.bin ftp> 150 TID=4636437270621265830 SIZE=0 SHA256=e3b0c44298fc1c149afb4c8996fb92 427ae41e4649b934ca495991b7852b855 [RD] 100% 0/0 bytes retries=0 0.016s 226 Transfer complete. </pre>
C	Active download showing PORT 200 and successful RETR	<pre> ftp> PORT 200 PORT command successful. ftp> RETR sample.bin ftp> 150 TID=8223698022025339485 SIZE=0 SHA256=e3b0c44298fc1c149afb4c8996fb92 427ae41e4649b934ca495991b7852b855 [RD] 100% 0/0 bytes retries=0 1.019s 226 Transfer complete. </pre>
D	Client/server SHA-256 comparison	<pre> ● quile@LAPTOP-NIMBN78N:~/hybrid-ftp-system\$ sha256sum storage/client_files/sample.bin e0a3982530d95ceac403a320b44edb3d3260f12dadf172ff2b077b107146812f storage/client_files/sample.bin ● quile@LAPTOP-NIMBN78N:~/hybrid-ftp-system\$ sha256sum storage/server_files/sample.bin e0a3982530d95ceac403a320b44edb3d3260f12dadf172ff2b077b107146812f storage/server_files/sample.bin </pre>
E	Two concurrent client sessions in the server log	<pre> ● quile@LAPTOP-NIMBN78N:~/hybrid-ftp-system\$ python3 server/tcp_control/tcp_server.py [2026-07-25 08:17:18] [INFO] [SERVER] TCP Control Engine listening on port 2121... [ACTIVE SESSIONS] FD CLIENT USER MODE TRANSFER 4 127.0.0.1:41794 - NONE IDLE [2026-07-25 08:17:26] [INFO] [FD 4] Session initialized for 127.0.0.1:41794 [2026-07-25 08:17:37] [INFO] [FD 4] Command received: USER admin [2026-07-25 08:17:46] [INFO] [FD 4] Command received: PASS 123456 [2026-07-25 08:17:46] [INFO] [FD 4] User 'admin' authenticated successfully. [ACTIVE SESSIONS] FD CLIENT USER MODE TRANSFER 4 127.0.0.1:41794 admin NONE IDLE [ACTIVE SESSIONS] FD CLIENT USER MODE TRANSFER 4 127.0.0.1:41794 admin NONE IDLE 5 127.0.0.1:57688 - NONE IDLE [2026-07-25 08:18:06] [INFO] [FD 5] Session initialized for 127.0.0.1:57688 [2026-07-25 08:18:12] [INFO] [FD 5] Command received: USER admin [Client] Connecting to 127.0.0.1:2121... 220 Welcome to RD-FTP Server (Hybrid TCP/UDP) ===== HYBRID FTP CLIENT CLI - READY FOR COMMANDS ===== [Auth/Session]: USER, PASS, QUIT [Data Mode] : PASV, PORT, TYPE [Directory] : PWD, CWD, CDUP, MKD, RMD [File/Data] : LIST, NLST, RETR, STOR, APPE, STOU, DELE, RNFR, RNTD, ABOR [Info/Meta] : STAT, SIZE, MDTM, HASH ===== ftp> USER admin 331 User name okay, need password. </pre>

		<pre>[Client] Connecting to 127.0.0.1:2121... 220 Welcome to RDT-FTP Server (Hybrid TCP/UDP) ===== HYBRID FTP CLIENT CLI - READY FOR COMMANDS ===== [Auth/Session]: USER, PASS, QUIT [Data Mode] : PASV, PORT, TYPE [Directory] : PWD, CWD, CDUP, MKD, RMD [File/Data] : LIST, NLST, RETR, STOR, APPE, STOU, DELE, RNFR, RNTD, ABOR [Info/Meta] : STAT, SIZE, MDTM, HASH ===== ftp> USER admin 331 User name okay, need password. ftp> PASS 123456 230 User logged in, proceed.</pre>
F	Fault-injection or retransmission statistics	<pre>quile@LAPTOP-NIMBN78N:~/hybrid-ftp-system\$ export RDT_LOSS_RATE=0.10 quile@LAPTOP-NIMBN78N:~/hybrid-ftp-system\$ export RDT_CORRUPTION_RATE=0.10 quile@LAPTOP-NIMBN78N:~/hybrid-ftp-system\$ python3 server/tcp_control/tcp_server.py [2026-07-25 08:23:50] [INFO] [SERVER] TCP Control Engine listening on port 2121... [ACTIVE SESSIONS] FD CLIENT USER MODE TRANSFER 4 127.0.0.1:45666 - NONE IDLE [2026-07-25 08:23:56] [INFO] [FD 4] Session initialized for 127.0.0.1:45666 [2026-07-25 08:24:00] [INFO] [FD 4] Command received: USER admin [2026-07-25 08:24:03] [INFO] [FD 4] Command received: PASS 123456 [2026-07-25 08:24:03] [INFO] [FD 4] User 'admin' authenticated successfully. [ACTIVE SESSIONS] FD CLIENT USER MODE TRANSFER 4 127.0.0.1:45666 admin NONE IDLE [2026-07-25 08:24:14] [INFO] [FD 4] Command received: PASV [2026-07-25 08:24:14] [INFO] [FD 4] Command received: STOR bigfile.bin SIZE=1048576 SHA256=7d9bb0b41380d7a7e940a693db806128b5a83f13073505495d68739a103e2066 [ACTIVE SESSIONS] FD CLIENT USER MODE TRANSFER 4 127.0.0.1:45666 admin PASSIVE ACTIVE [ACTIVE SESSIONS] FD CLIENT USER MODE TRANSFER 4 127.0.0.1:45666 admin NONE IDLE quile@LAPTOP-NIMBN78N:~/hybrid-ftp-system\$ python3 client/tcp_client/control_client.py [Client] Connecting to 127.0.0.1:2121... 220 Welcome to RDT-FTP Server (Hybrid TCP/UDP) ===== HYBRID FTP CLIENT CLI - READY FOR COMMANDS ===== [Auth/Session]: USER, PASS, QUIT [Data Mode] : PASV, PORT, TYPE [Directory] : PWD, CWD, CDUP, MKD, RMD [File/Data] : LIST, NLST, RETR, STOR, APPE, STOU, DELE, RNFR, RNTD, ABOR [Info/Meta] : STAT, SIZE, MDTM, HASH ===== ftp> USER admin 331 User name okay, need password. ftp> PASS 123456 230 User logged in, proceed. ftp> STOR bigfile.bin 227 Entering Passive Mode (127,0,0,1,143,108) 150 TID=12740532324704879355 SIZE=1048576 SHA256=7d9bb0b41380d7a7e940a693db806128b5a83f13073505495d68739a103e2066 [RDT] 100% 1048576/1048576 bytes retries=266 137.203s 226 Transfer complete.</pre>

7.4 Clean-Machine Demo Procedure

Terminal 1:
python server/tcp_control/tcp_server.py

Terminal 2:
python client/tcp_client/control_client.py


```
USER admin
PASS 123456
STOR sample.bin
PORT
RETR sample.bin
NLST
QUIT
```

Fault-injection demo (PowerShell): set `RDТ_LOSS_RATE=0.10`, `RDТ_CORRUPTION_RATE=0.05`, and `RDТ_DUPLICATE_RATE=0.05` before starting the server and client. The CLI prints progress, retry count, and duration.

Place `sample.bin` in `storage/client_files` before starting the client. The client automatically enters Passive mode when no mode has been selected. Enter `PORT` explicitly to demonstrate Active mode.

Conclusion

The implementation satisfies the central Hybrid FTP requirement: a reliable application-layer data protocol built directly over UDP while TCP remains responsible for control and session state. Stop-and-Wait provides bounded flow, deterministic ordering, and straightforward recovery. CRC32 protects each datagram; SHA-256 protects the complete transferred object. The modular boundary between FTP and RDТ allows the protocol to be tested independently and integrated in either data direction.

The primary performance limitation is the one-packet window. Throughput is approximately $\text{payload_size} / \text{RTT}$ when no loss occurs. A future extension could implement Go-Back-N or Selective Repeat with adaptive windows, while preserving the existing packet codec, transfer ID, checksum, and integrity workflow.